

Project Proposal

1. Project Title: Flickpond — A Video Sharing and Streaming Platform

2. Project Members

Name	Email
Jiang Yibai	e1583165@u.nus.edu
Zhang Ji	zhangji@u.nus.edu
Yang Dongwei	yangdongwei@u.nus.edu
Lu Jingxing	lujingxing@u.nus.edu
Ibrahim Mammadov	ibrahimmammadov@u.nus.edu

3. Overview

Media users and Small content teams often rely on separate tools to upload, process, store, and share video content, resulting in duplicated work, inconsistent playback formats, and limited control over content access. Flickpond aims to provide a single web-based platform where users can upload and manage their own videos, process them asynchronously into adaptive HLS formats, and share them with controlled visibility. The platform will also allow users to save and view references to selected YouTube videos through the provider's official API and embedded player, without downloading or storing third-party audiovisual content.

Flickpond is a web-based application that enables users to upload their own videos, transcode them into adaptive streaming formats, and watch them online with seamless playback. In addition, the platform allows users to paste a link from external video platforms, analyse the video metadata, and either stream the content directly or download it for offline viewing. The project will be developed using Agile methodologies, with a strong emphasis on DevSecOps practices—integrating security, continuous integration, and continuous delivery throughout the software development lifecycle.

Flickpond will use a modular monolithic web application with a separate asynchronous video-processing worker. The core application will contain clearly separated modules for identity and access management, video catalogue, upload management, playback, moderation, and analytics. A message queue will coordinate background transcoding tasks, while a relational database will manage application metadata and object storage will hold original and processed video files. Specific implementation tools will be selected after evaluating team expertise, compatibility, and available deployment resources. The component responsibilities and deployment structure will be described in the General Architecture section.

Key benefits of Flickpond include:

- Integrated Video Workflow: Flickpond brings video upload, background processing, controlled sharing, and playback into a single platform, reducing the need for separate tools and manual hand-offs.

- **Content Freedom:** Users retain ownership of their uploaded content with flexible privacy controls, and can import videos from external platforms without being locked into a single ecosystem.
- **Adaptive Playback:** Uploaded videos will be processed into multiple quality levels, allowing the player to adjust playback quality according to network conditions and device capabilities.
- **Cross-Platform Accessibility:** The platform supports video import from major third-party services (e.g., YouTube), enabling users to centralise their video consumption.
- **Security by Design:** DevSecOps principles are embedded throughout the pipeline, with automated SAST (SonarQube), DAST (OWASP ZAP), container vulnerability scanning (Trivy), and Infrastructure as Code (Terraform) ensuring a secure and auditable system.
- **Scalable Video Processing:** Time-consuming video-processing tasks will run asynchronously outside the main web application, allowing processing capacity to be adjusted independently as workload changes.

Overall, Flickpond will serve as a practical demonstration of modern software engineering principles—Agile development, microservices architecture, DevSecOps automation, and cloud-native deployment—while delivering a functional and user-friendly video platform.

4. Functional Requirements

4.1 User Management

This module handles user registration, authentication, and profile management, providing secure access control to the platform.

- The system will allow users to register with a unique email address, username, and password.
- Users must verify their email address before their account is activated.
- Registered users can log in with their credentials and remain authenticated through a time-limited session.
- Users can request a time-limited password reset link through their registered email address.
- Users can view and update their profile information, including their display name and avatar.
- Role-based access control will restrict Users to their own content, allow Content Moderators to review reported videos, and allow Administrators to manage user accounts and platform-wide access.
- Administrators can view, deactivate, and reactivate user accounts.

4.2 Video Upload and Management

This module enables authenticated users to upload, organise, and manage their own video content and its visibility.

- Users can upload video files in supported formats, with the file type and size validated before processing.
- Interrupted video uploads can be resumed without restarting from the beginning.

- Users can view the progress and current status of each upload.
- The system will allow users to add metadata to uploaded videos, including title, description, category, and tags.
- The system will allow users to set privacy levels for each video: Public (visible to all), Unlisted (accessible via link only), or Private (visible only to the uploader).
- The system will provide a user dashboard where creators can view their uploaded videos, view counts, and engagement metrics.
- The system will allow content moderators to review flagged content and remove videos that violate platform policies.
- Video owners can edit or delete their own uploaded videos.

4.3 Video Processing and Adaptive Playback

This module processes uploaded videos asynchronously and prepares them for adaptive playback.

- A video-processing job will be created automatically after a video is uploaded successfully.
- A separate worker will process video jobs in the background without blocking the main application.
- Each video will move through defined processing states, such as Pending, Processing, Ready, and Failed.
- Successfully processed videos will provide multiple playback quality levels.
- The system will adjust playback quality according to the viewer's network conditions.
- Video owners can view the processing status and will be notified when processing succeeds or fails.
- Failed processing jobs can be retried without requiring the video to be uploaded again.

4.4 External Video References

This module allows users to save references to selected YouTube videos and view them through the provider's official services.

- The system will allow users to paste a URL from supported third-party platforms (e.g., YouTube, Bilibili, Vimeo).
- The system will analyse the URL to extract video metadata, including title, thumbnail, duration, uploader, and available formats.
- Users can preview available metadata, such as title, thumbnail, channel, and duration, before saving the reference.
- The system will allow users to choose to either stream the video directly from the source or download a copy to the platform.
- The system will support downloading videos in user-selectable quality formats.
- The system displays a clear error when a video is unavailable, restricted, deleted, or cannot be retrieved.

4.5 Video Discovery and Playback

This module enables users to find and view videos that they are authorised to access.

- The system will display a homepage featuring recently uploaded and popular videos.
- Users can search available videos by title, description, category, or tags.
- Users can filter or sort search results by upload date, popularity, or relevance.
- A video detail page displays the player, title, description, owner, upload date, and view count.
- The system will record and display view counts for each video.
- The system records a view when playback reaches the agreed validation threshold
- Users may mark videos as favourites if the Could Have scope is reached.

4.6 Video Download

This module enables users to download videos from the platform.

- The system will allow users to download their own uploaded videos in their original format.
- For videos imported from third-party platforms, the system will allow download in selected quality formats.
- The system will generate a secure, time-limited download link for each download request.
- The system will restrict download permissions based on video privacy settings.

4.7 Moderation and Basic Analytics

This module supports basic content governance and provides evidence of platform usage.

- Users can report a video by selecting a reason and providing an optional comment.
- Content Moderators can review reported videos and record a moderation decision.
- Moderators can hide or restore videos without permanently deleting audit information.
- Administrators can view summary counts for users, videos, views, reports, and processing outcomes.
- The system can present a simple list or chart of popular videos if the Could Have scope is reached.

4.8 Scope Priorities

- Must Have: Authentication and RBAC; owned-video upload and management; asynchronous processing with status and retry; adaptive playback; browse and search; basic reporting and moderation; automated build, test, security checks, and staging deployment.
- Should Have: Email verification and password reset; selected YouTube references through official services; view counts and an administrative summary dashboard.
- Could Have: Favourites, comments, richer charts, and owner download of the original uploaded file.

- Out of Scope: Downloading, storing, or transcoding third-party audiovisual content; live streaming; personalised recommendations; subscriptions; native mobile applications; multi-region deployment; and production-scale container orchestration.

5. Non-Functional Requirements

5.1 Performance: In the staging environment, 90% of non-video application requests should complete within 2 seconds under a test load of 50 concurrent users. For a standard demonstration video and network, playback should begin within 5 seconds after selection. Test conditions and results will be recorded.

5.2 Reliability and Error Handling: Upload or processing failures must produce a clear user-visible state without losing the video record. Failed processing jobs must be retryable, and repeating the same job must not create conflicting video records or duplicate published outputs.

5.4 Security: All user passwords must be hashed using bcrypt. All API endpoints must require JWT authentication except for public video viewing. All video uploads must be scanned for malware. All container images must be scanned for vulnerabilities (Trivy) before deployment. The CI/CD pipeline must enforce quality gates—SAST (SonarQube) with zero critical vulnerabilities, and DAST (OWASP ZAP) with zero high-severity findings before production deployment.

5.5 Maintainability and Testability: The modular monolith and worker must have documented responsibilities and interfaces. Core service and workflow logic should achieve at least 70% automated unit-test line coverage, while critical end-to-end scenarios must have repeatable integration tests.

5.6 Usability and Compatibility: Core workflows must work on the latest stable versions of two major desktop browsers and remain usable at common tablet and desktop widths. Validation and processing errors must provide concise recovery guidance.

5.7 Observability: The application and worker must provide health status, structured logs with correlation identifiers, and enough processing-job information to diagnose failed uploads, queueing, processing, and playback preparation during testing and demonstration.

6. Effort Estimates

The project will run as four two-week calendar sprints. Each of the five members will contribute exactly 10.0 person-days across the project, averaging 2.5 person-days per sprint and producing a total effort of 50.0 person-days. Work will be performed collaboratively through pairing, peer review, integration, testing, and shared sprint ceremonies rather than as five isolated feature streams.

Iteration	Duration	Activities	Deliverables
1. Project Foundation & Identity	2 weeks	Sprint Planning & backlog prioritisation (Must/Should/Could/Out); use-case, domain, logical architecture & data modelling; Git repo setup, branching strategy, code-review workflow; automated build/lint/unit-test/security-check pipeline; implement registration, login, JWT session, RBAC, profile viewing.	Prioritised Product Backlog; Sprint 1 backlog with estimates & DoD; architecture & design artefacts; working staging deployment with auth + RBAC; pipeline with build, lint, test & security checks; review & retrospective records.
2. Owned-Video Upload & Management	2 weeks	Backlog refinement & Sprint 2 planning; detailed design for upload, metadata, ownership, visibility & processing status; implement file-type/size validation, resumable upload with progress, metadata management; implement Public/Unlisted/Private visibility; enforce owner/admin edit/delete permissions; create video record with Pending state after upload.	Updated backlog & design models; working resumable upload with validation & progress; working video catalogue with metadata, ownership & visibility; verified RBAC for video access; automated test results; demonstrable staging increment; review & retrospective records.
3. Processing, Playback, Discovery & External Reference	2 weeks	Sprint 3 planning; implement async video-processing worker with job coordination; implement video lifecycle states (Pending/Processing/Ready/Failed) & safe retry; prepare multi-quality HLS playback & adaptive streaming; implement browse, search, filtering, detail view & view-count; implement YouTube metadata API & embedded player (reference only, no download/store of third-party AV content); apply & document design patterns (State, Strategy, Adapter, Observer).	Working end-to-end upload – process -playback workflow; retry & failure-handling; browse/search/detail & view-count; YouTube reference via official API/embed; design patterns documented; test evidence (end-to-end, integration, external failure); review & retrospective records.

4. Moderation, QA, Deployment & Final Demo	2 weeks	Backlog review & closure; implement video reporting (reason + comment); implement moderator queue, decisions, hide/restore with audit preservation; admin summary dashboard (users/videos/views/reports/processing); complete SAST/DAST/container scans; performance test (50 concurrent, response time, playback start); observability validation (health/logs/tracing); achieve $\geq 70\%$ line coverage; regression testing; staging deployment; prepare demo script, report & evidence.	Fully integrated Flickpond MVP on staging; reporting, moderation, admin dashboard; final test & coverage reports; complete CI/CD pipeline evidence; complete Agile artefacts (backlogs, goals, reviews, retrospectives, DoD); complete analysis/design traceability; final report, demo script, known-limitations list; final review & retrospective.
--	---------	--	--